```
it "can create doubles" do
  twin = double(first_name: "Paul", weight: 100)
  expect(twin.first_name).to eq("Paul")
end
```

The assertion in the last line of the snippet is true because the stub has been preset to respond to the name message with "Paul." This spec is a bad way to use stubs. You've set up a nice little tautology and haven't learned anything about any larger system around this test.

The double method takes an optional hash argument, the keys being messages the stub will respond to and the values being the return values of those messages. These are stubbed methods, meaning that failure to use them will not trigger an error. Alternatively, you can use the same allow and expect methods on the double that you can on any other object. The following are equivalent:

```
twin = double(first_name: "Paul", weight: 100)
twin = double
allow(twin).to receive(first_name).and_return("Paul")
allow(twin).to receive(weight).and_return(100)
```

By default RSpec doubles are "strict," meaning that if you call the stub with a method that is not in the hash argument, RSpec will return an error. If that's not the behavior you want, RSpec provides the as_null_object method, which instead returns the double itself for methods not in the hash argument. Using as_null_object makes sense in the case where there's a large number of potential methods to be stubbed but where the values make so little difference that specifying them reduces the test's readability.

Null-object test doubles automatically spy on all methods, which is useful enough when using spies that RSpec provides spy(name) as an alternative method of writing double(name).as_null_object.

A common problem when using doubles is for the double and the object it is standing in for to drift apart. If the method name or signature changes in the real object, a test that only interacts with a double might not know, and no longer provides an accurate mirror of the real code. For this case, when you know your double needs to mimic a specific object, RSpec provides the concept of a *verifying* double. A verifying double checks to see whether messages passed to the double are real methods in the application. RSpec has a few methods to allow you to declare what to verify the double against:

```
instance_twin = instance_double("User")
instance_twin = instance_double(User)
class_twin = class_double("User")
class_twin = class_double(User)
object_twin = object_double(User.new)
```